MODELARE ȘI SIMULARE

Săptămâna 1. Introducere: sisteme, modele, simulare

Suport de curs · Anul III, Calculatoare și Tehnologia Informației
Ș.l. dr. ing. Cătălin Iapă

Obiective

1. să definiți noțiunile de sistem, model și simulare și să dați exemple;
2. să descrieți un sistem prin entități, atribute, stare, evenimente și activități;
3. să clasificați un model: static/dinamic, determinist/stocastic, cu timp și stare continue sau discrete;
4. să enumerați etapele unui studiu de simulare;
5. să explicați diferența dintre verificare și validare;
1. să dați exemple de utilizări actuale ale simulării.

Cuvinte cheie: *sistem, entitate, atribut, stare, eveniment, activitate, model, simulare, model stocastic, verificare, validare, emergență, digital twin*

1. De ce modelăm și simulăm?

1.1. O problemă de pornire

Un magazin online se așteaptă ca de Black Friday traficul să crească de zece ori. Trebuie să decidă câte servere închiriază: dacă închiriază prea puține, site-ul cade exact în ziua cea mai importantă a anului; dacă închiriază prea multe, plătește degeaba. Experimentul direct – „să încercăm Black Friday de câteva ori" – este imposibil.

Situații asemănătoare apar peste tot: o primărie vrea să știe dacă un sens giratoriu va reduce ambuteiajele, o autoritate de sănătate vrea să estimeze efectul închiderii școlilor asupra unei epidemii, un meteorolog vrea să prezică vremea de mâine. Toate au câteva lucruri în comun:

- sistemul are **multe componente** care interacționează (mașini, oameni, cereri, mase de aer);
- **aleatorul** joacă un rol (momentele de sosire, contactele dintre oameni, defectele);
- ne interesează **evoluția în timp** (ce se întâmplă la vârf, peste o săptămână, mâine);
- **experimentul direct** pe sistemul real este imposibil, prea scump, prea lent sau periculos;
- vrem să răspundem la întrebări de tipul **„ce-ar fi dacă…?"**.

Soluția comună este să construim o **copie simplificată** a sistemului în calculator și să experimentăm pe ea. Aceasta este, pe scurt, ideea modelării și simulării.

1.2. Sistemul și vocabularul de bază

Un **sistem** este o mulțime de componente care interacționează între ele și cu mediul înconjurător pentru a îndeplini o funcție (Holban, cap. 1). Exemple: o bancă, un procesor, o rețea de calculatoare, un oraș, un organism. Un aspect important este că **granița sistemului o alegem noi**, în funcție de întrebarea pe care o avem: dacă studiem cozile de la o bancă, clienții fac parte din sistem, iar traficul din oraș face parte din mediu. Mediul influențează sistemul prin **intrări** (de exemplu sosirea clienților), iar sistemul produce **ieșiri** (de exemplu timpul de așteptare).

Pentru a descrie un sistem folosim un vocabular comun, pe care îl vom utiliza tot semestrul (Tabelul 1).

Tabelul 1. Vocabularul de bază, ilustrat pe un ghișeu bancar și pe un server web

Concept	Definiție	Ghișeu bancar	Server web
Entitate	un obiect al sistemului care ne interesează	clientul	cererea HTTP
Atribut	o proprietate a unei entități	ora sosirii, tipul operației	momentul sosirii, dimensiunea răspunsului
Resursă	o entitate care servește alte entități și poate fi ocupată	casierul	nucleu de procesor, conexiune la baza de date
Stare	variabilele care descriu sistemul la un moment dat	clienți la coadă; casier liber/ocupat	cereri în așteptare; thread-uri ocupate
Eveniment	întâmplare instantanee care schimbă starea	sosire, început servire, plecare	sosire cerere, răspuns trimis
Activitate	acțiune cu durată, între două evenimente	servirea unui client	procesarea unei cereri

Starea merită o atenție specială. Ea este informația **minimă** care, împreună cu intrările viitoare, determină complet evoluția viitoare a sistemului. La un ghișeu nu trebuie să știm cine a venit la ora 9:03; ne ajunge câți clienți sunt la coadă și dacă funcționarul este ocupat. Alegerea variabilelor de stare este prima și una dintre cele mai importante decizii de modelare.

Distincția dintre **eveniment** și **activitate** este baza simulării cu evenimente discrete (săptămâna 5): evenimentul este instantaneu și schimbă starea (sosirea unui client crește coada cu 1), iar activitatea are durată și este delimitată de două evenimente (servirea începe cu evenimentul „început servire” și se termină cu „plecare”).

De reținut

Exercițiu: descrieți sistemul „cantina la prânz”. Entități: studenții (atribute: ora sosirii, meniul ales). Resurse: casele de marcat, mesele. Stare: numărul de studenți la coadă, mesele ocupate, casele libere/ocupate. Evenimente: sosire, început și sfârșit plată, ocupare și eliberare masă. Activități: plata, mâncatul.

1.3. Ce este un model?

Un **model** este o descriere abstractă și simplificată a unui sistem, construită pentru a răspunde la o anumită întrebare. Modelul păstrează doar elementele esențiale pentru acea întrebare și ignoră restul. Poate lua forma unor ecuații, a unor diagrame, a unor reguli sau a unui program – dar nu se identifică niciodată cu sistemul pe care îl descrie. Pictorul René Magritte a ilustrat ideea desenând o pipă cu inscripția „Ceci n’est pas une pipe” („Aceasta nu este o pipă”): desenul este doar reprezentarea unei pipe, la fel cum „harta nu este teritoriul” (A. Korzybski).

Statisticianul George Box a formulat ideea în fraza devenită celebră: **„All models are wrong, but some are useful”** (toate modelele sunt greșite, dar unele sunt utile). Orice model simplifică, deci este „greșit” într-o

măsură oarecare; întrebarea corectă este dacă modelul este **suficient de bun** pentru decizia pe care trebuie s-o luăm.

Folosim modelele pentru patru scopuri: **descriere** (organizăm ce știm despre sistem), **înțelegere** (aflăm de ce se comportă sistemul într-un anumit fel), **predicție** (estimăm ce se va întâmpla) și **control / optimizare** (alegem cea mai bună variantă).

Un model bun depinde de întrebare. El trebuie să fie suficient de simplu ca să poată fi înțeles, implementat și verificat, dar suficient de detaliat ca să surprindă ce contează pentru decizie. Relația dintre detaliu și utilitate nu este liniară: primele detalii adăugate aduc mult, apoi câștigul scade, iar costul (date, timp de dezvoltare, risc de erori) crește rapid. Un model prea detaliat cere date pe care nu le avem și devine greu de verificat. Principiul, atribuit lui Einstein, este: „totul ar trebui făcut cât mai simplu posibil, dar nu mai simplu de atât".

Același sistem poate avea mai multe modele bune. Traficul pe un bulevard poate fi descris ca un **fluid** (densitate și viteză, prin ecuații), ca un **automat celular** (drumul împărțit în celule, mașinile avansează după reguli simple) sau printr-un **model cu agenți** (fiecare șofer are destinație și comportament propriu). Alegerea depinde de întrebare.

1.4. Ce este simularea?

Simularea este tehnica de a realiza experimente cu ajutorul calculatorului, folosind un model care descrie comportamentul unui sistem real pe un interval de timp (Holban, cap. 1). Simularea imită evoluția sistemului, pas cu pas sau eveniment cu eveniment, și produce „instantanee" ale sistemului în diverse momente și condiții, din care extragem apoi statistici și concluzii.

Modelul și simularea nu sunt sinonime: **modelul este descrierea** sistemului (ca o rețetă de bucătărie), iar **simularea este execuția** modelului în timp (ca gătitul după rețetă). Un model se construiește o dată și poate fi folosit în mii de simulări, cu parametri și scenarii diferite.

Prin analogie cu experimentele biologice *in vivo* (pe organisme vii) și *in vitro* (în laborator), experimentele pe calculator se numesc *in silico*. Calculatorul devine un laborator virtual în care experimentul poate fi repetat, oprit, accelerat sau modificat oricât. Din acest motiv, știința computațională este considerată al treilea pilon al științei, alături de teorie și experiment.

1.5. Când folosim simularea – și când nu

Simularea este potrivită atunci când:

- nu există o soluție analitică (sistemul este prea complex, cu multe interacțiuni, reguli și aleator);
- experimentul pe sistemul real este imposibil, prea scump, periculos sau lent;
- vrem să comparăm mai multe variante („3 servere sau 4?", „semafor de 60 s sau 90 s?");
- timpul trebuie comprimat (ani de funcționare în câteva secunde) sau fenomene foarte rapide trebuie „încetinite";
- sistemul nu există încă (proiectarea unei fabrici, a unui procesor, a unei rețele);
- vrem să instruim oameni fără risc (simulatoare de zbor, de conducere).

Un exemplu din informatică: arhitecții de procesoare folosesc simulatoare precum **gem5**, care rulează programe reale pe un procesor virtual simulat ciclu cu ciclu. Simularea este de mii de ori mai lentă decât hardware-ul real, dar mult mai ieftină decât un prototip greșit.

La fel de important este să știm când **nu** folosim simularea (după Banks și colaboratorii): când problema se rezolvă cu bun-simț sau cu o formulă simplă; când este mai ieftin să experimentăm direct; când costul studiului depășește beneficiul deciziei; când nu avem date și nici nu le putem estima; când nu avem timp și oameni pentru verificare și validare; când se așteaptă de la simulare rezultate imposibile (certitudini, „optimul garantat").

De reținut

Simularea nu găsește singură soluția optimă, ci oferă tabloul rezultatelor pentru variantele analizate (Holban, cap. 1). Nu oferă certitudini, ci estimări cu marjă de eroare, și nu poate repara un model greșit („garbage in, garbage out").

1.6. Soluție analitică sau simulare?

Un sistem poate fi studiat prin experiment pe sistemul real sau pe un model; modelul poate fi fizic (o machetă) sau matematic; iar un model matematic poate fi rezolvat fie **analitic** (cu formule exacte), fie prin **simulare** (Law, cap. 1).

Un exemplu de soluție analitică este formula pentru coada cu un singur ghișeu, notată M/M/1 (sosiri aleatoare de tip Poisson, durate de servire exponențiale, un server). Timpul mediu petrecut de un client în sistem este:

$$W = 1 / (\mu - \lambda)$$

unde λ este rata medie de sosire, iar μ rata medie de servire. Pentru λ = 8 clienți pe oră și μ = 10 clienți pe oră obținem W = 0,5 ore, adică **30 de minute**. Formula spune mult: dacă λ se apropie de μ, timpul crește enorm (pentru λ = 9, W = 60 de minute). Dar ea este valabilă doar în ipoteze stricte. O bancă reală are mai multe ghișee, vârfuri de sosiri, operații scurte și lungi, clienți care renunță și un program limitat – pentru un astfel de sistem nu mai există o formulă simplă, iar simularea devine instrumentul natural.

Tabelul 2. Soluție analitică vs simulare

Criteriu	Soluție analitică	Simulare
Rezultat	exact (formulă)	estimat, cu o marjă de eroare
Complexitate acceptată	mică – ipoteze restrictive	mare – aproape orice reguli
Efort de calcul	foarte mic	de la mic la foarte mare
Înțelegere	arată direct cum depinde rezultatul de parametri	cere multe rulări și analiză
Aleator	tratat prin probabilități	generat explicit, rulări repetate

1.7. O scurtă istorie

În 1777, Buffon a estimat numărul π aruncând un ac pe o suprafață cu linii paralele – primul „experiment Monte Carlo". Metoda Monte Carlo propriu-zisă a apărut în anii 1940 (Ulam, von Neumann, Metropolis), odată cu primele calculatoare electronice. În 1961 apare **GPSS** (G. Gordon, IBM), primul limbaj de simulare popular, iar în 1967 **Simula 67** (O.-J. Dahl și K. Nygaard, Norvegia). Simula a fost creat pentru simularea

sistemelor și a introdus conceptele de **clasă** și **obiect**: programarea orientată pe obiecte s-a născut, așadar, din simulare, iar ideile au fost preluate de C++ și Java. Astăzi simularea se face în biblioteci precum SimPy, în medii specializate precum AnyLogic și, tot mai des, în combinație cu inteligența artificială.

De reținut

Simulăm când experimentul real este imposibil, scump, lent sau periculos. Sistem = componente care interacționează, cu o graniță aleasă de noi. Model = descriere simplificată, pentru o întrebare. Simulare = execuția modelului în timp. Simularea dă estimări, nu certitudini.

2. Ce fel de modele există?

Clasificarea modelelor nu este un exercițiu teoretic: tipul modelului determină metoda de rezolvare și uneltele. Ea ne obligă să ne întrebăm dacă timpul, aleatorul sau spațiul contează în problema noastră și ne permite să comunicăm precis („un model stocastic, cu evenimente discrete").

2.1. Modele fizice, matematice și computaționale

Un **model fizic** este o machetă sau un prototip: o aripă testată în tunelul aerodinamic sau macheta golfului San Francisco (Bay Model, 1957), construită pentru studiul mareelor. Un **model matematic** este format din ecuații și relații (legea lui Newton, formula cozii M/M/1, ecuațiile unei epidemii). Un **model computațional** este un program care implementează reguli (agenți, evenimente, celule) și adesea nu are soluție analitică. În practică, un model trece prin mai multe forme: sistemul real → model conceptual (în cuvinte și diagrame, cu ipotezele scrise explicit) → model matematic sau logic → model computațional → simulare.

2.2. Elementele unui model matematic

Orice model matematic dinamic are aceleași elemente (Holban, cap. 6): **intrările** u(t), pe care le controlăm sau le observăm; **ieșirile** y(t), pe care le măsurăm; **starea** x(t), formată din funcții de timp care reține informația minimă despre sistem; **parametrii** θ, constante care „potrivesc" modelul pe un sistem anume (de exemplu rata de servire). Ieșirile depind atât de intrări, cât și de **starea inițială** $x(t_0)$: aceleași intrări aplicate unui sistem aflat în stări diferite produc răspunsuri diferite.

2.3. Static și dinamic

Într-un **model static** timpul nu joacă un rol: calculăm un rezultat pentru o situație sau pentru multe scenarii (riscul unui proiect estimat din 10 000 de scenarii aleatoare). Un **model dinamic** descrie evoluția în timp (coada de la ghișeu pe parcursul unei zile, numărul de infectați săptămână de săptămână).

2.4. Determinist și stocastic

Un **model determinist** nu conține elemente aleatoare: aceleași date de intrare produc întotdeauna același rezultat, deci o singură rulare este suficientă. Un **model stocastic** conține variabile aleatoare: aceleași date de intrare produc rezultate diferite. Consecința practică este majoră: un model stocastic trebuie rulat de multe ori, rezultatul este o **distribuție**, iar concluziile se raportează cu un **interval de încredere**. Treizeci de rulări independente sunt un punct de pornire folosit des. Variabilitatea nu este o „eroare" a simulării, ci o proprietate a sistemului real; la sistemele informatice ne interesează adesea tocmai cazurile extreme (percentilele 95 sau 99 ale latenței), nu doar media.

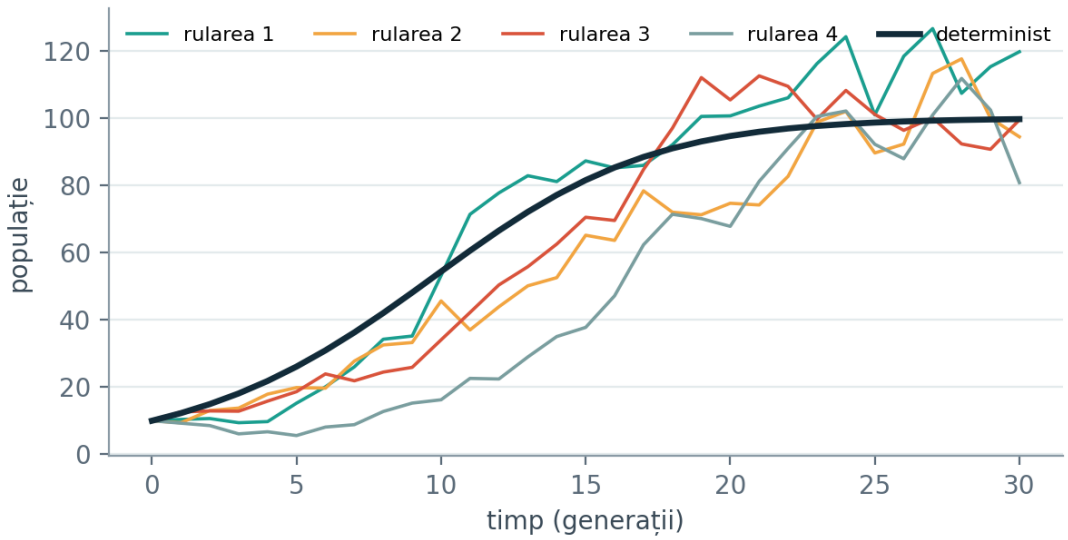


Figura 1. Același model de creștere a unei populații, în varianta deterministă (linia neagră) și în patru rulări ale variantei stocastice

2.5. Timp și stare: continuu sau discret

În modelele cu **timp continuu**, starea poate fi observată în orice moment și se descrie de obicei prin ecuații diferențiale; în modelele cu **timp discret**, starea este definită doar la pașii t = 0, 1, 2, … și se descrie prin recurențe de forma x(t+1) = f(x(t)). Independent de timp, **starea** poate fi **continuă** (valori reale care se schimbă lin: temperatura, nivelul apei) sau **discretă** (valori numărabile care se schimbă prin salturi: numărul de clienți la coadă, un server pornit/oprit). Combinând cele două axe obținem patru familii de modele (Tabelul 3).

Tabelul 3. Cele patru familii de modele, după timp și stare

	Stare continuă	Stare discretă
Timp continuu	ecuații diferențiale (EDO): răcire, SIR, pradă–prădător – săptămâna 7	evenimente discrete (DES): cozi, fabrici, rețele – săptămânile 4–5
Timp discret	recurențe / ecuații cu diferențe: populații anuale, modele lunare	automate celulare, lanțuri Markov – săptămânile 3 și 8

Un aspect important: **discret sau continuu este o alegere a modelatorului**, nu o proprietate a sistemului. Traficul poate fi descris ca un fluid continuu sau ca mașini individuale; o populație de milioane – ca un număr real în ecuații sau ca indivizi numărați unul câte unul. Cu un număr mare de entități, aproximarea continuă este adesea bună și mult mai ieftină.

2.6. Descriptiv, predictiv, prescriptiv

După scop, un model poate fi **descriptiv** (ce se întâmplă și de ce? – de ce stația de biciclete din campus rămâne goală dimineața), **predictiv** (ce se va întâmpla? – câte biciclete vor fi la fiecare stație la ora 9) sau **prescriptiv** (ce ar trebui să facem? – câte biciclete să mutăm noaptea și unde). Modelele prescriptive combină de obicei simularea cu optimizarea sau cu evaluarea mai multor variante.

2.7. Top-down, bottom-up și emergența

Un model **top-down** (agregat) descrie direct mărimi globale, prin ecuații pentru medii – de exemplu modelul SIR al unei epidemii, cu trei ecuații diferențiale și doi parametri. Este rapid și are puțini parametri, dar presupune o populație omogenă. Un model **bottom-up** (individual) descrie fiecare individ și regulile lui – de exemplu un model cu agenți în care fiecare persoană își infectează vecinii cu o anumită probabilitate. Comportamentul global **rezultă** din interacțiuni; modelul este mai costisitor, dar surprinde diversitatea, spațiul și rețeaua de contacte.

```
# TOP-DOWN (model SIR):  dS/dt = -b*S*I,  dI/dt = b*S*I - g*I,  dR/dt = g*I

# BOTTOM-UP: fiecare persoană este un agent
for p in persoane:
    if p.stare == "infectat":
        for v in p.vecini():
            if v.stare == "susceptibil" and random() < b:
                v.stare = "infectat"
        if random() < g:
            p.stare = "vindecat"
```

Un **comportament emergent** este un tipar global care apare din interacțiuni locale simple, fără să fie programat explicit în regulile indivizilor: stolurile de păsări (fiecare pasăre urmează trei reguli simple – modelul Boids, pe care îl veți programa la laborator) sau „ambuteiajul fantomă”, în care o frânare mică se propagă înapoi și creează un blocaj. Modelele bottom-up pot surprinde astfel de fenomene, pe care ecuațiile agregate nu le arată.

2.8. Scări de spațiu și de timp

Orice sistem poate fi privit la mai multe scări: **micro** (molecule, mașini individuale, celule, pachete de rețea), **meso** (elemente de fluid, grupuri de mașini, țesuturi, fluxuri TCP) și **macro** (clima, traficul unui oraș, organe, încărcarea totală a rețelei). Adesea modelăm la o scară mai fină decât întrebarea: întrebarea este macro („se formează ambuteiaj?”), modelul este micro (reguli pentru fiecare mașină). Surprinzător, detaliile microscopice contează puțin pentru comportamentul macroscopic, cât timp respectăm legile de conservare și simetriile (Cernazean, curs 1) – de aceea automate celulare foarte simple pot reproduce curgerea fluidelor (săptămâna 8).

Și scările de timp diferă enorm: un procesor lucrează în nanosecunde, o rețea în milisecunde, o coadă la ghișeu în minute, o epidemie sau clima în luni și ani. Scara timpului decide pasul și orizontul simulării; un model nu poate acoperi eficient toate scările, deci alegem scara potrivită întrebării.

2.9. Harta metodelor din curs

Tabelul 4. Metodele studiate în acest curs

Metodă	Ideea	Săpt.	Tip de model
Monte Carlo	estimăm prin eșantionare aleatoare repetată	S2	static, stocastic
Lanțuri Markov	stări și probabilități de tranziție	S3	discret
Teoria cozilor	formule pentru sisteme de servire	S4	analitic
Evenimente discrete	starea sare la momentele evenimentelor	S5	dinamic, stocastic
Ecuații diferențiale	schimbări continue ale mărimilor agregate	S7	continuu

Metodă	Ideea	Săpt.	Tip de model
Automate celulare	celule cu reguli locale, pe o rețea	S8	discret, spațial
Modele cu agenți	indivizi autonomi care interacționează	S9	bottom-up
Modele din date / AI	modelul se învață din date sau din simulări	S10	data-driven

O primă regulă de alegere a metodei: dacă evoluția în timp nu contează → Monte Carlo. Dacă timpul contează și starea se schimbă prin salturi la momente anume → evenimente discrete. Dacă schimbările sunt continue: când contează indivizii și spațiul → agenți sau automate celulare; altfel → ecuații diferențiale. În practică metodele se combină des.

2.10. Proprietățile unui model bun și greșeli frecvente

Un model bun este **valid** (reproduce realitatea acolo unde o cunoaștem, pentru întrebarea pusă), **simplu**, **transparent** (ipotezele și datele sunt scrise și pot fi verificate), **reproductibil** (altcineva, cu același cod și aceleași date, obține aceleași rezultate), **robust** (concluziile nu se schimbă radical la mici variații ale parametrilor) și **util** (răspunde la timp la o întrebare care contează). În comunitatea de modelare există două filozofii: **KISS** („Keep It Simple, Stupid” – pornim de la cel mai simplu model și adăugăm detalii doar dacă sunt necesare) și **KIDS** („Keep It Descriptive, Stupid” – pornim de la un model bogat, apropiat de date, și simplificăm). Pentru început, KISS este recomandarea noastră.

Greșelile frecvente sunt: modelul prea complicat de la început; modelul prea simplu, căruia îi lipsește tocmai mecanismul care contează; date de intrare inventate sau nepotrivite; concluzii trase dintr-o singură rulare a unui model stocastic; lipsa verificării (bug-uri care produc rezultate „plauzibile”); confundarea modelului cu realitatea. Un sfat practic: scrieți lista ipotezelor **înainte** de a scrie codul.

De reținut

Clasificăm modelele pe mai multe axe: fizic / matematic / computațional; static / dinamic; determinist / stocastic; timp continuu / discret; stare continuă / discretă; descriptiv / predictiv / prescriptiv; top-down / bottom-up. Discret sau continuu este alegerea modelatorului. Modelele stocastice cer multe rulări.

3. Cum se face un studiu de simulare

Un studiu de simulare este un **proiect**, nu doar un program. Codul este de obicei partea cea mai mică; timpul se duce pe înțelegerea problemei, pe date, pe verificare și validare și pe explicarea rezultatelor.

3.1. Etapele unui studiu

Adaptând schemele din Law, Banks și Holban (1.2), un studiu de simulare are zece etape, parcurse iterativ:

1. **Formularea problemei**: care este întrebarea, cine folosește răspunsul, ce măsuri de performanță (timp de așteptare, cost, utilizare), ce alternative comparăm, ce orizont de timp, ce precizie. „Să simulăm fabrica” nu este o întrebare.

2. **Modelul conceptual**: entități, resurse, graniță, evenimente, reguli și lista ipotezelor simplificatoare – validat cu experții sistemului.
3. **Colectarea datelor**: din măsurători, loguri, baze de date, experți; din ele extragem distribuții. Principiul „garbage in, garbage out".
4. **Implementarea**: într-un limbaj general (Python), o bibliotecă de simulare (SimPy, Mesa) sau un mediu specializat (AnyLogic, Arena, Modelica).
5. **Verificarea**: am construit corect modelul?
6. **Validarea**: am construit modelul corect?
7. **Proiectarea experimentelor**: scenarii, număr de replicări, durata rulărilor, perioada de încălzire.
8. **Rularea și analiza**: medii cu intervale de încredere, comparații statistice între variante (detaliate în săptămâna 6).
9. **Documentarea**: ipoteze, date, versiunea codului și seed-urile, rezultate și limite.
10. **Decizia**: rezultatele explicate pe limba beneficiarului; uneori modelul rămâne în uz și este actualizat continuu (digital twin).

3.2. Verificare și validare

Aceasta este una dintre cele mai importante distincții din curs. **Verificarea** răspunde la întrebarea „**am construit corect modelul?**" – programul face ce spune modelul conceptual? Caută bug-uri și erori de logică. **Validarea** răspunde la întrebarea „**am construit modelul corect?**" – modelul reprezintă suficient de bine realitatea pentru scopul nostru? Caută ipoteze greșite. Un model poate fi perfect verificat și totuși invalid. Dacă verificarea găsește o problemă, ne întoarcem la cod; dacă validarea găsește o problemă, ne întoarcem la ipoteze, adică la modelul conceptual.

Tehnici simple și eficiente: **cazurile extreme** (rata de sosire zero → coada trebuie să fie goală), **compararea cu formule** cunoscute într-un caz simplu (M/M/1), **urmărirea pas cu pas** (trace) și **animația**, **compararea cu date istorice**.

3.3. Studiu de caz: biciclete partajate

Exemplul, preluat din cartea lui A. B. Downey (*Modeling and Simulation in Python*, cap. 2–3), este dezvoltat și la laborator. Două campusuri, Olin și Wellesley, au câte o stație de biciclete, cu 12 biciclete în total (10 la Olin și 2 la Wellesley la început). În fiecare minut, cu probabilitatea p_1 un student pleacă cu bicicleta de la Olin la Wellesley, iar cu probabilitatea p_2 invers. Dacă stația de plecare este goală, studentul pleacă nemulțumit.

Formularea problemei. Cum evoluează numărul de biciclete la fiecare stație și câți clienți găsesc stația goală (măsura de performanță) pe o perioadă de 60 de minute? Cum influențează rata p_1 numărul de nemulțumiți?

Tabelul 5. Modelul conceptual al sistemului bike share

Element	Descriere
Stare	numărul de biciclete la Olin și la Wellesley (suma este mereu 12)
Timp	discret, pași de câte 1 minut
Evenimente	plecare Olin → Wellesley (probabilitate p_1); plecare Wellesley → Olin (probabilitate p_2)
Regulă	dacă stația de plecare este goală, clientul nu primește bicicletă și este contorizat ca nemulțumit

Element	Descriere
Ipoteze	durata drumului este neglijabilă; cel mult o plecare în fiecare sens pe minut; p_1 și p_2 constante
Tip de model	dinamic, stocastic, timp discret, stare discretă

Implementarea în Python are sub treizeci de linii:

```
import random

def muta(stare, de_la, la):
    if stare[de_la] == 0:              # stație goală
        stare["nemultumiti"] += 1
        return
    stare[de_la] -= 1
    stare[la] += 1

def pas(stare, p1, p2):                # un minut
    if random.random() < p1:
        muta(stare, "olin", "wellesley")
    if random.random() < p2:
        muta(stare, "wellesley", "olin")

def simuleaza(p1, p2, minute=60, seed=None):
    random.seed(seed)                  # reproductibilitate
    stare = {"olin": 10, "wellesley": 2, "nemultumiti": 0}
    istoric = [stare["olin"]]
    for t in range(minute):
        pas(stare, p1, p2)
        istoric.append(stare["olin"])
    return istoric, stare["nemultumiti"]
```

Rezultate. O rulare cu $p_1 = 0{,}5$ și $p_2 = 0{,}33$ arată cum bicicletele „curg” spre Wellesley, deoarece fluxul în acel sens este mai mare (Figura 2). Suma rămâne mereu 12 – o primă verificare. Rulând același model de mai multe ori, cu seed-uri diferite, obținem traiectorii foarte diferite (Figura 3): modelul este stocastic, iar o singură rulare nu permite concluzii.

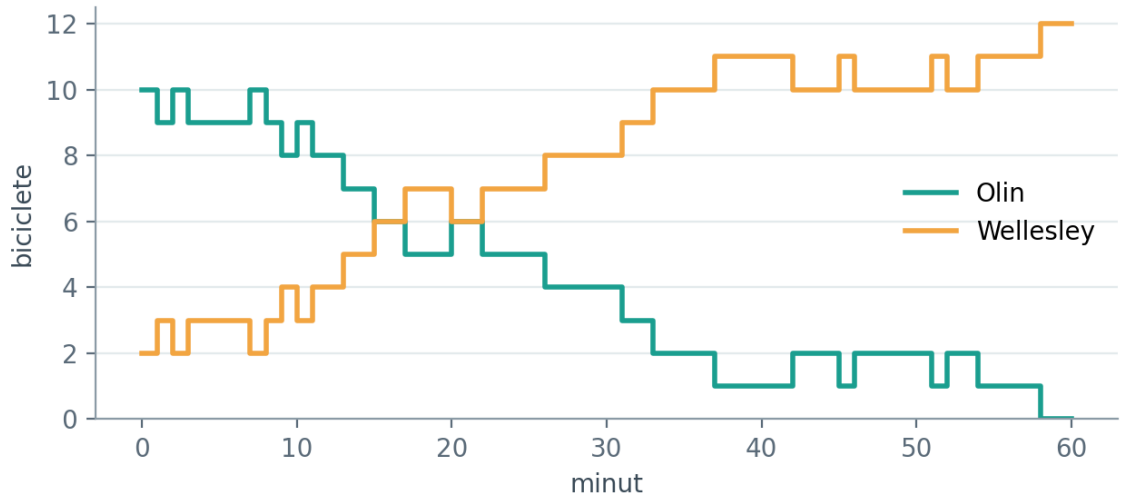


Figura 2. O rulare a modelului bike share ($p_1 = 0{,}5$, $p_2 = 0{,}33$)

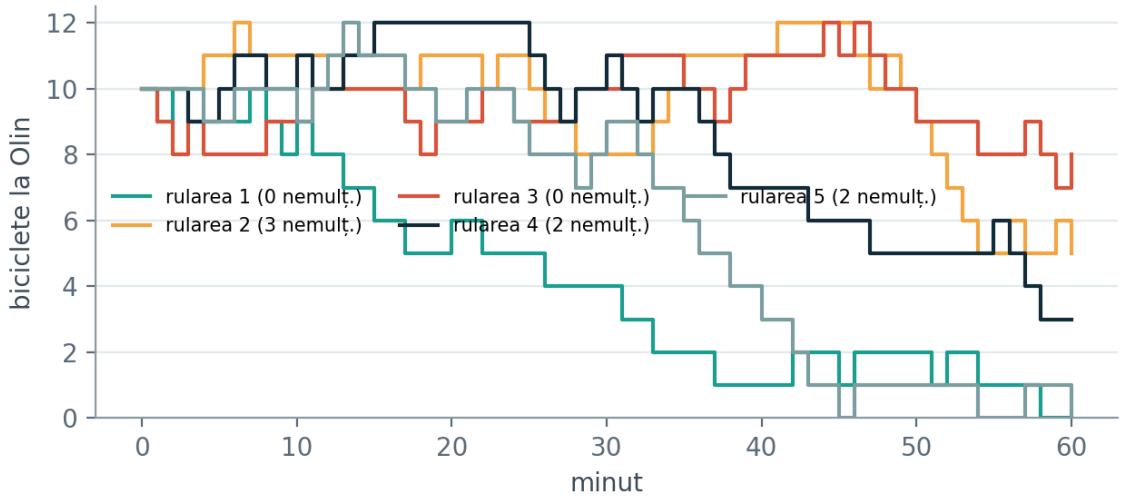


Figura 3. Cinci rulări ale aceluiași model: numărul de biciclete la Olin

Experiment. Variind p_1 între 0 și 1 și rulând de 200 de ori pentru fiecare valoare, obținem numărul mediu de nemulțumiți (Figura 4). Curba are formă de U: când p_1 este mic se golește Wellesley, când p_1 este mare se golește Olin, iar minimul apare când fluxurile sunt aproximativ echilibrate ($p_1 \approx p_2$). De aici rezultă o idee de politică: rebalansarea periodică a bicicletelor – exact ce fac sistemele reale. Acesta este un exemplu simplu de **analiză de sensibilitate**.

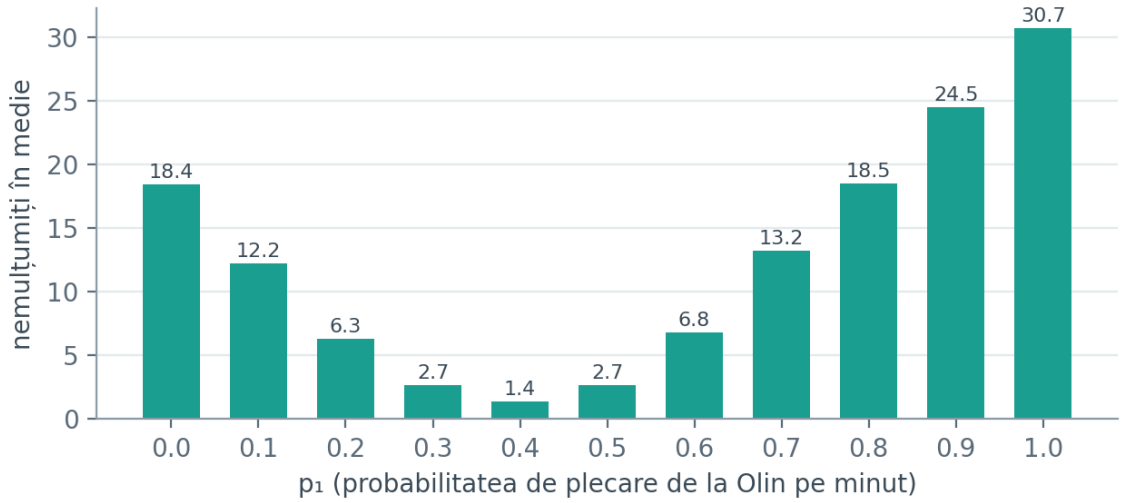


Figura 4. Numărul mediu de clienți nemulțumiți în funcție de p_1 (200 de replicări pentru fiecare valoare, $p_2 = 0{,}33$)

Verificare. Suma bicicletelor este constantă; nicio stație nu are valori negative; pentru $p_1 = p_2 = 0$ starea nu se schimbă; pentru $p_1 = 1$ și $p_2 = 0$, Olin se golește după 10 minute, iar apoi apare câte un nemulțumit pe minut – 50 în total, rezultat calculabil de mână; același seed produce exact aceeași rulare. **Validare.** Comparăm cu date reale de utilizare (la laborator: date din Londra), estimăm p_1 și p_2 pe intervale orare și verificăm dacă modelul reproduce orele în care stațiile se golesc. Ipoteza probabilităților constante este evident nerealistă și va fi îmbunătățită la laborator.

Un **raport de simulare** bun conține, pe scurt: întrebarea și măsura de performanță, lista ipotezelor și sursele datelor, rezultatele (medii cu intervale de încredere, grafice simple, comparații) și limitele modelului.

3.4. Frontiera: unde se folosește simularea azi

Prognoza meteo cu AI. În 2024, modelul GenCast (Google DeepMind, publicat în *Nature*) a fost mai precis decât sistemul european de referință ENS pe 97,2% din cele 1 320 de ținte evaluate și produce o prognoză pe 15 zile în aproximativ 8 minute pe un singur cip TPU. A fost antrenat pe aproximativ 40 de ani de date de reanaliză – produse la rândul lor cu simulări numerice. AI-ul nu elimină simularea, ci învață din ea.

Digital twins. Conform National Academies (2023), un digital twin este un set de construcții informaționale virtuale care imită structura, contextul și comportamentul unui sistem natural, tehnic sau social, este actualizat dinamic cu date din sistemul fizic, are capacitate predictivă și informează decizii care aduc valoare. Esențială este bucla bidirecțională: date de la senzori → model → predicții → decizii → acțiuni asupra sistemului fizic.

Roboți antrenați în simulator. Roboții învață prin încercare și eroare (învățare prin întărire) în simulatoare precum MuJoCo sau NVIDIA Isaac Lab, în mii de copii paralele pe GPU. Provocarea este transferul în realitate (*sim-to-real*); o soluție este randomizarea domeniului – variem parametrii simulării ca politica învățată să devină robustă. Aceeași idee stă la baza datelor sintetice pentru mașinile autonome.

Agenți generativi. În 2023, Park și colaboratorii au creat un orășel virtual cu 25 de agenți conduși de modele de limbaj, care își planificau ziua, discutau, își aminteau evenimente și au organizat „spontan” o petrecere. Asemenea agenți sunt folosiți acum în simulări sociale, dar validarea rămâne problema centrală (Törnberg și Larooij, 2025): un comportament care pare realist nu este neapărat corect.

Simularea este folosită curent în fabrici și logistică, sănătate, energie, transport, IT și cloud (capacitatea centrelor de date, testarea sistemelor distribuite), jocuri și film. În acest curs vom folosi Python (NumPy, SciPy, Matplotlib), SimPy pentru evenimente discrete și Mesa pentru agenți; mediile specializate (AnyLogic, NetLogo) vor fi doar prezentate. Conceptele nu depind însă de limbaj.

De reținut

Un studiu de simulare are 10 etape, parcurse iterativ; cele mai neglijate sunt formularea problemei și validarea. Verificare = am construit corect modelul? Validare = am construit modelul corect? Cazurile extreme și legile de conservare sunt teste de verificare simple și eficiente. Un model stocastic se judecă pe baza multor rulări.

Glosar RO – EN

Română	English
sistem	system
mediu	environment
entitate	entity
atribut	attribute
resursă	resource
stare	state

Română	English
eveniment	event
activitate	activity
model	model
simulare	simulation
soluție analitică	analytical solution
model static / dinamic	static / dynamic model
model determinist / stocastic	deterministic / stochastic model
timp continuu / discret	continuous / discrete time
stare continuă / discretă	continuous / discrete state
simulare cu evenimente discrete	discrete-event simulation (DES)
model descriptiv / predictiv / prescriptiv	descriptive / predictive / prescriptive model
comportament emergent	emergent behaviour
model conceptual	conceptual model
ipoteză simplificatoare	simplifying assumption
verificare	verification
validare	validation
replicare	replication
analiză de sensibilitate	sensitivity analysis
interval de încredere	confidence interval
geamăn digital	digital twin
învățare prin întărire	reinforcement learning

Întrebări de autoevaluare

1. Care dintre următoarele descrie cel mai bine un model?

a) O copie completă și exactă a unui sistem real

b) O descriere simplificată a unui sistem, construită pentru a răspunde la o anumită întrebare

c) Un program care rulează în timp real

d) Un set de date măsurate pe un sistem

2. Starea unui sistem este:

a) Lista tuturor evenimentelor care au avut loc până acum

b) Media rezultatelor obținute în mai multe simulări

c) Informația minimă care, împreună cu intrările viitoare, determină evoluția viitoare a sistemului

d) Mulțimea parametrilor constanți ai modelului

3. Într-un model al unui ghișeu bancar, care dintre următoarele este un **eveniment**?

a) Sosirea unui client

b) Servirea unui client, care durează 4 minute

c) Coada de clienți

d) Funcționarul de la ghișeu

4. În care situație simularea este cea mai puțin potrivită?

a) Dimensionarea unei secții de urgențe care nu există încă

b) Calculul dobânzii simple pentru un depozit bancar

c) Evaluarea efectului unui nou sens giratoriu

d) Estimarea latenței unui server la un vârf de trafic

5. Riscul unui proiect este estimat generând 10 000 de scenarii aleatoare pentru costuri și calculând media. Modelul este:

a) static și stocastic

b) dinamic și determinist

c) dinamic și stocastic

d) static și determinist

6. O coadă la ghișeu este simulată eveniment cu eveniment, cu sosiri și plecări la momente oarecare. Numărul de clienți din sistem este o stare:

a) continuă, în timp discret

b) discretă, în timp continuu

c) continuă, în timp continuu

d) discretă, în timp discret

7. Traiectoria unei mingi aruncate, calculată cu legile lui Newton, fără perturbații, este un model:

a) static și stocastic

b) dinamic, determinist, cu stare continuă

c) dinamic, stocastic, cu stare discretă

d) static și determinist

8. Verificarea unui model de simulare răspunde la întrebarea:

a) Am construit corect modelul? (programul face ce spune modelul conceptual)

b) Am construit modelul corect? (modelul reprezintă realitatea)

c) Este modelul cel mai simplu posibil?

d) Au fost colectate suficiente date?

9. Simularea unei bănci nu are niciun bug, dar prezice cozi de 3 ori mai mici decât cele reale, pentru că presupune că toți clienții fac operații scurte. Problema este de:

a) verificare

b) validare

c) implementare

d) generare a numerelor aleatoare

10. Care este diferența esențială dintre un digital twin și o simulare obișnuită?

a) Digital twin-ul folosește grafică 3D

b) Digital twin-ul este actualizat continuu cu date din sistemul real și informează decizii

c) Digital twin-ul nu conține elemente aleatoare

d) Digital twin-ul rulează doar pe supercalculatoare

11. Care dintre următoarele este un model **bottom-up**?

a) Trei ecuații diferențiale pentru fracțiunile de populație S, I, R

b) O formulă pentru timpul mediu de așteptare într-o coadă

c) Un model în care fiecare persoană este un agent care își poate infecta vecinii

d) O regresie liniară între temperatură și consumul de energie

12. Care este un exemplu de comportament emergent?

a) Un program care afișează „Hello, world”

b) Răcirea unei cafele după o curbă exponențială

c) Calculul exact al unei integrale

d) Un ambuteiaj care apare fără accident, din frânări mici ale șoferilor

13. De ce un model stocastic trebuie rulat de mai multe ori?

a) Pentru că prima rulare este întotdeauna greșită

b) Pentru că astfel dispare aleatorul din model

c) Pentru că fiecare rulare dă un rezultat diferit, iar concluziile cer statistici pe multe rulări

d) Pentru că programele de simulare conțin bug-uri

14. Care este primul pas al unui studiu de simulare?

a) Formularea problemei și a obiectivelor

b) Scrierea codului

c) Rularea experimentelor

d) Alegerea mediului de simulare

15. Limbajul Simula 67, creat pentru simularea sistemelor, este cunoscut în informatică pentru că:

a) a fost primul limbaj compilat

b) a introdus programarea funcțională

c) a fost primul limbaj pentru calcul paralel

d) a introdus conceptele de clasă și obiect (primul limbaj orientat pe obiecte)

16. Coadă M/M/1: sosesc în medie $\lambda = 8$ clienți pe oră, iar ghișeul servește în medie $\mu = 10$ clienți pe oră. Folosind $W = 1/(\mu - \lambda)$, cât este timpul mediu petrecut în sistem, **în minute**?

(Răspuns numeric)

17. Aceeași coadă M/M/1, dar rata de sosire crește la $\lambda = 9$ clienți pe oră ($\mu = 10$). Cât este acum timpul mediu petrecut în sistem, **în minute**?

(Răspuns numeric)

18. În modelul bike share (10 biciclete inițial la Olin, 2 la Wellesley), cu $p_1 = 1$ și $p_2 = 0$, pe o durată de 60 de minute, câți clienți nemulțumiți ar trebui să raporteze un program corect?

(Răspuns numeric)

19. În același model bike share, cât trebuie să fie în orice moment suma bicicletelor de la cele două stații? (test de verificare prin conservare)

(Răspuns numeric)

20. Combinând cele două tipuri de timp (continuu / discret) cu cele două tipuri de stare (continuă / discretă), câte familii de modele obținem?

(Răspuns numeric)

Răspunsuri

1. b) – Modelul este o abstracție: păstrează doar ce este esențial pentru întrebarea pusă.

2. c) – Istoria completă nu este necesară; ajunge informația minimă pentru a continua evoluția.

3. a) – Evenimentul este instantaneu și schimbă starea; servirea este o activitate (are durată); coada ține de stare, iar funcționarul este o resursă/entitate.

4. b) – Pentru dobânda simplă există o formulă exactă și simplă; simularea ar fi risipă.

5. a) – Nu există evoluție în timp (static), iar scenariile sunt aleatoare (stocastic) – o simulare Monte Carlo.

6. b) – Numărul de clienți este întreg (stare discretă), iar evenimentele pot avea loc în orice moment (timp continuu).

7. b) – Poziția evoluează în timp (dinamic), nu există aleator (determinist), poziția și viteza sunt numere reale (stare continuă).

8. a) – Verificarea privește implementarea; validarea privește legătura cu realitatea.

9. b) – Codul este corect, dar o ipoteză a modelului nu corespunde realității: problemă de validare.

10. b) – Bucla cu sistemul real (date → model → decizii → sistem) definește un digital twin.

11. c) – Modelul bottom-up descrie indivizii și regulile lor; comportamentul global rezultă din interacțiuni.

12. d) – Tiparul global (ambuteiajul) nu este programat explicit, ci apare din interacțiuni locale.

13. c) – Rezultatul unui model stocastic este o distribuție; o singură rulare este doar o realizare a ei.

14. a) – Fără o întrebare precisă nu știm ce să includem în model și ce să măsurăm.

15. d) – Entitățile simulate au devenit obiecte, iar tipurile lor – clase; ideile au fost preluate de C++ și Java.

16. 30 – W = 1/(10 − 8) = 0,5 ore = 30 de minute.

17. 60 – W = 1/(10 − 9) = 1 oră = 60 de minute: o creștere de 12,5% a sosirilor dublează timpul.

18. 50 – Olin se golește după 10 minute; în celelalte 50 de minute fiecare client găsește stația goală: 50.

19. 12 – Nicio bicicletă nu apare și nu dispare: 10 + 2 = 12.

20. 4 – 2 × 2 = 4: EDO, evenimente discrete, recurențe, automate celulare / lanțuri Markov.

Bibliografie

- A. M. Law – Simulation Modeling and Analysis, ed. 6, McGraw Hill, 2024, cap. 1.
- J. Banks, J. S. Carson, B. L. Nelson, D. M. Nicol – Discrete-Event System Simulation, ed. 5, Pearson, 2010, cap. 1.
- A. B. Downey – Modeling and Simulation in Python, No Starch Press, 2023, cap. 1–3 (gratuită online: allendowney.github.io/ModSimPy).
- Ș. Holban – Tehnici de modelare și simulare, cap. 1 și 6 (materiale de curs).
- Materialele de curs Modelling and Simulation (secția engleză), cursul 1.
- National Academies of Sciences, Engineering, and Medicine – Foundational Research Gaps and Future Directions for Digital Twins, 2023.
- I. Price et al. – Probabilistic weather forecasting with machine learning, Nature, 2024.
- J. S. Park et al. – Generative Agents: Interactive Simulacra of Human Behavior, UIST 2023.
- P. Törnberg, M. Larooij – Validation is the central challenge for generative social simulation, Artificial Intelligence Review, 2025.
- Material realizat cu ajutorul Claude Fable 5.1 și Claude Opus 5.5